



Static Application Security Testing

Dibimbing-Shop

Comprehensive Security Analysis Report

REPORT ID

80406087-e19d-4feb-beb6-123fd1bb
9531

ASSESSMENT DATE

May 24, 2026

LINES OF CODE

15693

RISK RATING

CRITICAL

Table of Contents

1. Executive Summary	3
2. Assessment Overview	4
3. Vulnerability Summary	5
4. Critical Vulnerabilities	6
5. High Severity Vulnerabilities	7
6. Medium & Low Vulnerabilities	8
7. Conclusion	9

1 Executive Summary

Security Assessment

Comprehensive static analysis of Dibimbing-Shop codebase

84

TOTAL FINDINGS

15693

LINES OF CODE

45

FILES ANALYZED

CRITICAL

RISK RATING

Vulnerability Distribution

52

CRITICAL

30

HIGH

0

MEDIUM

2

LOW

⚠ **Critical Action Required:** This codebase contains 52 critical security vulnerabilities that require immediate remediation. These issues pose significant security risks and should be addressed as a priority.

2 Assessment Overview

2.1 Scan Details

Project	Dibimbing-Shop
Repository	https://github.com/rezafahmialviandy/Dibimbing-Shop
Branch	main
Scan Date	May 24, 2026
Report ID	80406087-e19d-4feb-beb6-123fd1bb9531
Lines of Code	15693
Files Analyzed	45

2.2 Vulnerability Breakdown

SEVERITY	COUNT	STATUS
CRITICAL	52	Requires Action
HIGH	30	Review Required
MEDIUM	0	Pass
LOW	2	Track

3 Vulnerability Summary

3.1 Top Vulnerability Types

VULNERABILITY TYPE	COUNT	SEVERITY
Unsanitized external input in SQL query	51	critical
Unsanitized user input in 'echo' function (XSS)	23	high
Cross-Site Scripting	7	high
Insecure direct object reference (IDOR) — record loader keyed on user input	2	low
SQL Injection	1	critical

3.2 Most Affected Files

FILE PATH	VULNERABILITIES	SEVERITY DISTRIBUTION
admin_area/user_profile.php	10	C:2 H:8
admin_area/edit_store.php	7	C:2 H:5
admin_area/edit_term.php	5	C:2 H:3
admin_area/edit_coupon.php	4	C:2 H:2
admin_area/edit_rel.php	4	C:3 H:1
admin_area/edit_cat.php	3	C:2 H:1
edit_about_us.php	3	H:3
customer/confirm.php	3	C:2 H:1
edit_contact_us.php	3	H:3
admin_area/edit_manufacturer.php	3	C:2 H:1

4 Critical Vulnerabilities

⚠ Immediate Action Required

The following 52 critical vulnerabilities require immediate remediation to prevent potential security breaches.

Unsanitized external input in SQL query

[admin_area/edit_contact_us.php](#)
[Line 148](#)
[CWE-89](#)

VULNERABLE CODE

```
$run_contact_us = mysqli_query($con,$update_contact_us);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/customer_delete.php](#)
[Line 26](#)
[CWE-89](#)

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_customer);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/product.php](#)
[Line 177](#)
[CWE-89](#)

VULNERABLE CODE

```
if (mysqli_query($con, $update_sql)) {
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/delete_term.php](#)
[Line 23](#)
[CWE-89](#)

VULNERABLE CODE

```
$run_term = mysqli_query($con,$delete_term);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_manufacturer.php](#)
[Line 23](#)
[CWE-89](#)

VULNERABLE CODE

```
$run_manufacturer = mysqli_query($con,$get_manufacturer);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

VULNERABLE CODE

```
$run_manufacturer = mysqli_query($con,$update_manufacturer);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/insert_user.php

Line 211

CWE-89

VULNERABLE CODE

```
$run_admin = mysqli_query($con,$insert_admin);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/delete_cat.php

Line 22

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_cat);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

functions/cart_body.php

Line 208

CWE-89

VULNERABLE CODE

```
$run_coupons = mysqli_query($con,$get_coupons);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/confirm.php

Line 27

CWE-89

VULNERABLE CODE

```
$run_payment = mysqli_query($con, $insert_payment);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/confirm.php

Line 30

CWE-89

VULNERABLE CODE

```
$run_customer_order = mysqli_query($con, $update_customer_order);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/insert_manufacturer.php

Line 116

CWE-89

VULNERABLE CODE

```
$run_manufacturer = mysqli_query($con,$insert_manufacturer);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/index.php

Line 530

CWE-89

VULNERABLE CODE

```
if (mysqli_query($con, $delete_query)) {
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/delete_manufacturer.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_manufacturer = mysqli_query($con,$delete_manufacturer);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/payment_delete.php

Line 25

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_payment);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/change_pass.php

Line 339

CWE-89

VULNERABLE CODE

```
$run_old_pass = mysqli_query($con,$sel_old_pass);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/change_pass.php

Line 369

CWE-89

VULNERABLE CODE

```
$run_pass = mysqli_query($con,$update_pass);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/delete_coupon.php

Line 23

CWE-89

VULNERABLE CODE


```
$run_delete = mysqli_query($con,$delete_coupon);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_about_us.php](#)[Line 154](#)[CWE-89](#)

VULNERABLE CODE

```
$run_about_us = mysqli_query($con,$update_about_us);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/user_delete.php](#)[Line 22](#)[CWE-89](#)

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_user);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/insert_store.php](#)[Line 179](#)[CWE-89](#)

VULNERABLE CODE

```
$run_store = mysqli_query($con,$insert_store);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/insert_rel.php](#)[Line 170](#)[CWE-89](#)

VULNERABLE CODE

```
$run_rel = mysqli_query($con,$insert_rel);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_rel.php](#)[Line 24](#)[CWE-89](#)

VULNERABLE CODE

```
$run_edit = mysqli_query($con,$edit_rel);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_rel.php](#)[Line 38](#)[CWE-89](#)

VULNERABLE CODE

```
$run_p = mysqli_query($con,$get_p);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_rel.php](#)[Line 217](#)[CWE-89](#)

VULNERABLE CODE

```
$run_rel = mysqli_query($con,$update_rel);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/insert_term.php](#)[Line 131](#)[CWE-89](#)

VULNERABLE CODE

```
$run_term = mysqli_query($con,$insert_term);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/order_delete.php](#)[Line 23](#)[CWE-89](#)

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_order);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/delete_p_cat.php](#)[Line 23](#)[CWE-89](#)

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_p_cat);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_store.php](#)[Line 25](#)[CWE-89](#)

VULNERABLE CODE

```
$run_store = mysqli_query($con,$get_store);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_store.php](#)[Line 207](#)[CWE-89](#)

VULNERABLE CODE

```
$run_store = mysqli_query($con,$update_store);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_cat.php

Line 22

CWE-89

VULNERABLE CODE

```
$run_edit = mysqli_query($con,$edit_cat);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_cat.php

Line 150

CWE-89

VULNERABLE CODE

```
$run_cat = mysqli_query($con,$update_cat);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/fetch_orders.php

Line 17

CWE-89

VULNERABLE CODE

```
$result = mysqli_query($con, $query);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/insert_enquiry.php

Line 96

CWE-89

VULNERABLE CODE

```
$run_enquiry = mysqli_query($con,$insert_enquiry);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

customer/delete_wishlist.php

Line 10

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_wishlist);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

SQL Injection

edit_coupon.php

Line 158

CWE-89

VULNERABLE CODE

```
$run_p = mysqli_query($con,$get_p);
```

DESCRIPTION

Tainted data flows from user input to SQL query execution without sanitization.

Unsanitized external input in SQL query

admin_area/delete_rel.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_rel);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/delete_store.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_store);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/user_profile.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_admin = mysqli_query($con,$get_admin);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/user_profile.php

Line 259

CWE-89

VULNERABLE CODE

```
$run_admin = mysqli_query($con,$update_admin);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

cart.php

Line 214

CWE-89

VULNERABLE CODE

```
$run_coupons = mysqli_query($con,$get_coupons);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/delete_enquiry.php

Line 22

CWE-89

VULNERABLE CODE

```
$run_delete = mysqli_query($con,$delete_enquiry);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

VULNERABLE CODE

```
$run_email = mysqli_query($con,$get_email);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/insert_coupon.php

Line 213

CWE-89

VULNERABLE CODE

```
$run_coupon = mysqli_query($con,$insert_coupon);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/insert_cat.php

Line 116

CWE-89

VULNERABLE CODE

```
$run_cat = mysqli_query($con,$insert_cat);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_coupon.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_edit = mysqli_query($con,$edit_coupon);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_coupon.php

Line 240

CWE-89

VULNERABLE CODE

```
$run_coupon = mysqli_query($con,$update_coupon);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_term.php

Line 23

CWE-89

VULNERABLE CODE

```
$run_term = mysqli_query($con,$get_term);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

admin_area/edit_term.php

Line 158

CWE-89

VULNERABLE CODE

```
$run_term = mysqli_query($con,$update_term);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[functions/change.php](#)[Line 24](#)[CWE-89](#)

VULNERABLE CODE

```
$run_qty = mysqli_query($con,$change_qty);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_enquiry.php](#)[Line 23](#)[CWE-89](#)

VULNERABLE CODE

```
$run_enquiry_type = mysqli_query($con,$get_enquiry_type);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

Unsanitized external input in SQL query

[admin_area/edit_enquiry.php](#)[Line 117](#)[CWE-89](#)

VULNERABLE CODE

```
$run_enquiry = mysqli_query($con,$update_enquiry);
```

DESCRIPTION

Using unsanitized data, such as user input or request data, or externally influenced data passed to a function, in SQL query exposes your application to SQL injection attacks. This vulnerability arises when externally controlled data is directly included in SQL statements without proper sanitatio...

5 High Severity Vulnerabilities

Cross-Site Scripting

edit_contact_us.php

Line 80

CWE-79

VULNERABLE CODE

```
<input type="text" name="contact_email" class="form-control" value="<?php echo $contact_email; ?>">
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Cross-Site Scripting

edit_contact_us.php

Line 92

CWE-79

VULNERABLE CODE

```
<input type="text" name="contact_heading" class="form-control" value="<?php echo $contact_heading; ?>">
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Cross-Site Scripting

edit_contact_us.php

Line 105

CWE-79

VULNERABLE CODE

```
<?php echo $contact_desc; ?>
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_manufacturer.php

Line 86

CWE-79

VULNERABLE CODE

```
<input type="text" name="manufacturer_name" class="form-control" value="<?php echo $m_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

customer/confirm.php

Line 356

CWE-79

VULNERABLE CODE

```
<p class="info-value amount-value">Rp <?php echo number_format($due_amount, 0, ',', '.'); ?></p>
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Cross-Site Scripting

functions.php

Line 284

CWE-79

VULNERABLE CODE

```
echo "<li class='page-item'><a class='page-link' href='shop.php?page=1' . (!empty($aPath) ? "&" . $aPath : '') . "'>First Page</a></li>";
```

DESCRIPTION

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_rel.php

Line 107

CWE-79

VULNERABLE CODE

```
<input type="text" name="rel_title" class="form-control" value="<?php echo $r_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_store.php

Line 92

CWE-79

VULNERABLE CODE

```
<input type="text" name="store_title" class="form-control" value="<?php echo $store_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_store.php

Line 110

CWE-79

VULNERABLE CODE

```

```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_store.php

Line 125

CWE-79

VULNERABLE CODE

```
<?php echo $store_desc; ?>
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_store.php

Line 140

CWE-79

VULNERABLE CODE

```
<input type="text" name="store_button" class="form-control" value="<?php echo $store_button; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_store.php

Line 152

CWE-79

VULNERABLE CODE

```
<input type="url" name="store_url" class="form-control" value="<?php echo $store_url; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_cat.php Line 83 **CWE-79**

VULNERABLE CODE

```
<input type="text" name="cat_title" class="form-control" value="<?php echo $c_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Cross-Site Scripting

edit_about_us.php Line 80 **CWE-79**

VULNERABLE CODE

```
<input type="text" name="about_heading" class="form-control" value="<?php echo $about_heading; ?>">
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Cross-Site Scripting

edit_about_us.php Line 95 **CWE-79**

VULNERABLE CODE

```
<?php echo $about_short_desc; ?>
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Cross-Site Scripting

edit_about_us.php Line 111 **CWE-79**

VULNERABLE CODE

```
<?php echo $about_desc; ?>
```

DESCRIPTION

Tainted data flows from user input to HTTP response without proper encoding.

Unsanitized user input in 'echo' function (XSS)

admin_area/user_profile.php Line 104 **CWE-79**

VULNERABLE CODE

```
<input type="text" name="admin_name" class="form-control" required value="<?php echo $admin_name; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/user_profile.php Line 117 **CWE-79**

VULNERABLE CODE

```
<input type="text" name="admin_email" class="form-control" required value="<?php echo $admin_email; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/user_profile.php Line 130 **CWE-79**

VULNERABLE CODE

```
<input type="text" name="admin_pass" class="form-control" required value="<?php echo $admin_pass; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/user_profile.php](#)[Line 142](#)[CWE-79](#)

VULNERABLE CODE

```
<input type="text" name="admin_country" class="form-control" required value="<?php echo $admin_country; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/user_profile.php](#)[Line 155](#)[CWE-79](#)

VULNERABLE CODE

```
<input type="text" name="admin_job" class="form-control" required value="<?php echo $admin_job; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/user_profile.php](#)[Line 168](#)[CWE-79](#)

VULNERABLE CODE

```
<input type="text" name="admin_contact" class="form-control" required value="<?php echo $admin_contact; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/user_profile.php](#)[Line 181](#)[CWE-79](#)

VULNERABLE CODE

```
<textarea name="admin_about" class="form-control" rows="3"> <?php echo $admin_about; ?> </textarea>
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/user_profile.php](#)[Line 195](#)[CWE-79](#)

VULNERABLE CODE

```

```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

[admin_area/edit_coupon.php](#)[Line 101](#)[CWE-79](#)

VULNERABLE CODE

```
<input type="text" name="coupon_title" class="form-control" value="<?php echo $c_title; ?>" >
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_coupon.php Line 151 CWE-79

VULNERABLE CODE

```
<option value="<?php echo $product_id; ?>"> <?php echo $product_title; ?> </option>
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_term.php Line 90 CWE-79

VULNERABLE CODE

```
<input type="text" name="term_title" class="form-control" value="<?php echo $term_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_term.php Line 103 CWE-79

VULNERABLE CODE

```
<?php echo $term_desc; ?>
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_term.php Line 116 CWE-79

VULNERABLE CODE

```
<input type="text" name="term_link" class="form-control" value="<?php echo $term_link; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

Unsanitized user input in 'echo' function (XSS)

admin_area/edit_enquiry.php Line 80 CWE-79

VULNERABLE CODE

```
<input type="text" name="enquiry_title" class="form-control" value="<?php echo $enquiry_title; ?>">
```

DESCRIPTION

Including unsanitized user input in responses can lead to cross-site scripting (XSS) attacks. This vulnerability exposes your application to malicious scripts injected by attackers, compromising user data and browser security.

6 Medium & Low Vulnerabilities

6.2 Low Severity (2 findings)

VULNERABILITY	FILE	LINE	CWE
Insecure direct object reference (IDOR) — record loader keyed on user input	admin_area/index.php	519	639, 862
Insecure direct object reference (IDOR) — record loader keyed on user input	details.php	68	639, 862

7 Conclusion

7.1 Assessment Summary

This Static Application Security Testing (SAST) analysis of **Dibimbing-Shop** scanned **15693** lines of code across **45** files and identified **84** security vulnerabilities.

Critical Findings: 52 critical vulnerabilities require immediate attention. These issues represent significant security risks that could lead to data breaches, unauthorized access, or system compromise.

High Priority: 30 high severity vulnerabilities should be addressed promptly to maintain a strong security posture.

7.2 Recommendations

- **Immediate Action:** Prioritize remediation of all 52 critical vulnerabilities identified in this report.
- **Short-term:** Address the 30 high severity findings within the next sprint cycle.
- **Code Review:** Implement peer code reviews with security focus to catch vulnerabilities early.
- **Continuous Scanning:** Integrate SAST scanning into your CI/CD pipeline for ongoing security monitoring.
- **Developer Training:** Provide secure coding training to prevent introduction of similar vulnerabilities.
- **Security Testing:** Complement SAST with DAST and penetration testing for comprehensive coverage.

7.3 Next Steps

1. Review and validate all findings with development team
2. Create remediation tickets for each vulnerability in issue tracking system
3. Assign owners and deadlines based on severity
4. Implement fixes following secure coding best practices
5. Perform verification scan after remediation
6. Update security documentation and runbooks